A C Primer (9): I/O and Files

Study notes for Systems Programming

Based on the SPC9-IO slide deck (16 slides; original by Ion Mandoiu and Laurent Michel, revised by M. Khan, J. Shi and W. Wei). Every slide is covered; added explanations, examples and pitfalls are marked as extra where they go beyond the slides.

What is inside

#	Topic	Source
1	Error handling with errno	slide 2
2	Files and directories	slide 3
3	The stdio library and the FILE stream model	slides 4-6
4	Opening and closing streams	slides 7-8
5	Character I/O: fgetc, fputc, getc, putc, ungetc, getchar, putchar	slides 9-11
6	Line I/O: fgets and fputs	slide 12
7	Formatted I/O: fscanf and fprintf	slide 13
8	Binary I/O: fread and fwrite	slide 14
9	Moving around a file: ftell and fseek	slide 15
10	feof and fflush	slide 16
11	Worked examples	extra
12	Common pitfalls, quick-reference table, self-check questions	extra
13	Slide-by-slide coverage checklist	

> Big picture: C does file I/O through the **stdio** library. You open a file to get a `FILE *`, pass that pointer to functions that read or write bytes at the current **position**, check every return value, and close the file when done. Everything in these notes is a variation on that pattern.

1. Error handling with errno

Slide 2

Most C library functions can **fail**. When they do, they report it through their **return value**, usually a special flag such as `-1`, `NULL` or `EOF`. Some functions also store a more specific **error code** in a global variable called `errno`.

Using errno

- Include the header: `#include <errno.h>`.
- Check the function's return value first. Only look at `errno` **after** a call has reported failure; a successful call is allowed to leave a stale or changed value there.
- To interpret the numeric code, check the **manual page** of the function (e.g. `man 3 fopen`, section ERRORS). Codes are named constants such as `ENOENT` (no such file) or `EACCES` (permission denied).
- To print a human-readable message use `void perror(const char *str);`. It prints your `str`, a colon, and the text for the current `errno` to stderr.

perror example

```
FILE *fp = fopen("missing.txt", "r");
if (fp == NULL) {
    perror("fopen");        // prints: fopen: No such file or directory
    return 1;
}
```

Multithreaded code

The slide advises: **avoid functions that set errno in multithreaded code, and prefer thread-safe versions when available**. The worry is that several threads share state hidden inside library functions. Examples of non-thread-safe library functions are `strtok` and `strerror`, whose thread-safe counterparts are `strtok_r` and `strerror_r`.

> **Good to know**
> Extra: on modern Linux/glibc, `errno` itself is implemented per thread, so each thread sees its own value. The classic danger is still real for functions that use shared static buffers, which is what the slide's advice protects against.

2. Files and directories

Slide 3

- A **file** is an object that stores information or data.
  - Files you create with an editor: `.c`, `.h`, Makefile, readme, ...
  - The executable generated by the compiler, and `gcc` itself.
  - Other devices, like the screen and the keyboard.
- In Linux, files are organised in **directories**. A directory can contain subdirectories and files. The top directory is `/` (the root).
- A **path** specifies the location of a file or directory in the file system, for example `/home/john`.

In Unix/Linux, everything is a file. Regular files, directories, and devices (terminal, keyboard, screen) are all accessed with the same read/write/open/close style of operations. That is why a program can read from the keyboard with the same functions it uses to read from a disk file.

Good to know
Extra: a path starting with `/` is **absolute** (from the root); any other path is **relative** to the program's current working directory. `.` means the current directory and `..` the parent directory.

3. The stdio library and the FILE stream model

Slides 4, 5 and 6

The stdio library (slide 4)

```
#include <stdio.h>
```

- Declares the `FILE` type and the function prototypes for file I/O.
- `FILE` is an **opaque type** (system dependent) used to operate on files. It is a structure, but **do not try to change it directly**. Always use library functions, through pointers of type `FILE *`.
- Defines the three **standard streams** `stdin`, `stdout` and `stderr`. They are created automatically when the program starts, and they are files too. Each one is a `FILE *`.
- The library is **linked automatically** by the compiler (no extra `-l` flag needed), but you **must include the header**.

Stream	Meaning	Default connected to
`stdin`	standard input	keyboard
`stdout`	standard output	screen
`stderr`	standard error (error messages)	screen

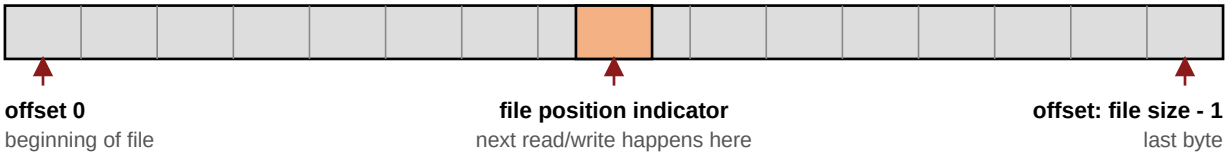
Files and the I/O API (slide 5)

- In C a file is simply a **sequential stream of bytes**. C does not know about records or lines in the file itself; text lines are just bytes with `'\n'` in them.
- The "f" family of functions (`fopen, fclose, fread, fgetc, fscanf, fprintf, ...`) are **C library functions**. They all use a `FILE *` to represent a file.
- The C library provides **buffering**: data is collected in memory and moved to or from the file in larger chunks, which is faster than one system call per byte.
  - This is why sometimes you do not see the output of `printf` immediately. The text is sitting in the buffer. `fflush` (section 10) forces it out.
- The slides also announce that a **second set of functions provided by the OS** (file descriptors, system calls) will be taught later. Stdio sits on top of those.

Good to know
Extra: typical buffering behaviour in glibc. **stdout** is line-buffered when connected to a terminal (flushed at each newline) and fully buffered when redirected to a file or pipe. **stderr** is unbuffered, so error messages appear immediately.

File as stream of bytes (slide 6)

A file is a sequence of bytes, one position indicator



- Before use, a file must be **opened**. Opening sets up a **position indicator** for reading and/or writing.
- Each read or write starts from the **current position** and then **moves the indicator** forward by the number of bytes transferred.
- Writing after the last byte **increases the file size**.
- The position indicator can also be moved explicitly with `fseek` (section 9).
- All open files are closed automatically when the program ends, but it is **good practice to close explicitly** as soon as you no longer need the file.

> **Watch out**
> Closing explicitly matters. A normal exit flushes and closes streams, but if the program crashes or calls `_exit`, buffered data can be lost. Open files are also a limited resource, so a long-running program that never closes them will eventually fail to open new ones.

4. Opening and closing streams

Slides 7 and 8

fopen (slide 7)

```
FILE* fopen(const char *filename, const char *mode);
```

- Opens the file named `filename` in `mode` as a stream of bytes.
- Returns a **pointer to FILE** on success, or **NULL** on failure (and `errno` is set).
- **Check return values!** Using a NULL `FILE *` crashes the program.

Mode	What it does (from the slide)	File must exist?	Truncated?
`"r"`	Reading mode	Yes	No
`"r+"`	Read and write	Yes	No
`"w"`	Writing mode; file is created or truncated to zero length	No (created)	Yes
`"w+"`	Read and write, but the file is created or truncated	No (created)	Yes
`"a"`	Append mode; the file is created if it does not exist	No (created)	No
`"a+"`	Read and append; created if it does not exist. Reading starts at the beginning, but writing is always done at the end	No (created)	No

> **Watch out**
> **"w" and "w+" destroy existing content** the moment the file is opened. If you only want to add to the end, use "a". The speaker note on slide 7 adds that in "a+" the **first write moves the file pointer to the end**, no matter where you were reading.

The standard open-and-check pattern

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    perror("data.txt");
    return 1;
}
```

fclose (slide 8)

```
int fclose(FILE *stream);
```

- Closes a stream. Pending buffered output is written first.
- Returns **0** if it worked, or **EOF** if there was a problem (and `errno` is set).

> **Good to know**
> Extra: for files you *wrote*, checking the return value of `fclose` is worthwhile, because a delayed write error (such as a full disk) can first show up when the last buffer is flushed.

5. Character I/O

Slides 9, 10 and 11

fgetc / fputc: one byte at a time (slide 9)

```
int fgetc(FILE *stream);
int fputc(int c, FILE *stream);
```

- Read or write **one (ASCII) character (8 bits)** at a time. This can be **slow for large files**.
- `fgetc` reads a character from the stream and returns it **as an unsigned char extended to int**. It returns **EOF** at end of file or on error.
- `fputc` writes the character it receives to the stream and returns the character just written, or **EOF** on error.

> **Watch out**
> **Store the result of fgetc in an int, not a char.** `EOF` is a negative int (usually -1). If you store it in a plain `char`, either a legitimate byte 0xFF is mistaken for EOF (signed char) or EOF is never detected (unsigned char). This is exactly why the return type is `int`.

Copy loop

```
int c;                                  // int, not char!
while ((c = fgetc(in)) != EOF)
    fputc(c, out);
```

getc / putc and ungetc (slide 10)

```
int getc(FILE *stream);
int putc(int c, FILE *stream);
int ungetc(int c, FILE *stream);
```

- `getc` and `putc` are the same as `fgetc`/`fputc` **except they may be implemented as macros**. A macro can evaluate its arguments more than once, so never pass an expression with side effects such as `getc(files[i++])`.
- **Use fgetc/fputc unless you have strong reasons not to.**
- `ungetc` **pushes the last read character back** onto the stream, where it is available for the next read operation. Only **one pushback is guaranteed**. It is useful for "peeking": read a character to decide what to do, then put it back if it belongs to the next token.

ungetc as a one-character lookahead (needs <ctype.h>)

```
// read an unsigned integer, stopping at the first non-digit
int c, value = 0;
while ((c = fgetc(fp)) != EOF && isdigit(c))
    value = value * 10 + (c - '0');
if (c != EOF)
    ungetc(c, fp);                      // give the non-digit back
```

Good to know
Extra: calling `fseek` on a stream discards any pushed-back character.

getchar / putchar (slide 11)

```
int getchar(void);        // same as fgetc(stdin)
int putchar(int c);       // same as fputc(c, stdout)
```

- `getchar` reads a character from **stdin**. It returns the character just read, or **EOF** on end-of-file or errors.
- `putchar` writes the character to **stdout** and returns the character just written, or **EOF** on errors.
- They are shortcuts for the common case of the standard streams. At a terminal, end-of-file on stdin is produced with Ctrl-D (Linux).

6. Line I/O: fgets and fputs

Slide 12

fgets

```
char *fgets(char *buf, int size, FILE *in);
```

- Reads the **next line** from `in` into the buffer `buf`.
- Halts at `'\n'` **or** after `size-1` characters have been read. A **NUL** (`'\0'`) is placed at the end, which is why at most size-1 characters fit.
- Returns the pointer `buf` if OK, or **NULL** otherwise (end of file with nothing read, or an error).
- **Do not use gets(char *)!** It has no size argument, so it cannot protect against **buffer overflow**. (It was removed from the C standard in C11.)

Watch out
If the line fits, `fgets` **keeps the newline** at the end of the buffer. If the line is longer than size-1, you get only the first part and the next call returns the rest. Strip the newline with `buf[strcspn(buf, "\n")] = '\0';` (needs).

fputs

```
int fputs(const char *str, FILE *out);
```

- Writes the string `str` to `out`, **stopping at '\0'** (the terminator itself is not written).
- Returns the number of characters written (per the slide) or **EOF** on error. Strictly, the C standard promises only a non-negative value on success.

> **Good to know**
> Extra: unlike `puts`, `fputs` does **not** add a newline. Include `\n` in your string if you want one.

Print a file line by line

```
char line[256];
while (fgets(line, sizeof line, fp) != NULL) {
    fputs(line, stdout);              // line still has its '\n'
}
```

7. Formatted I/O: fscanf and fprintf

Slide 13

```
int fscanf(FILE *stream, const char *format, ...);
int fprintf(FILE *stream, const char *format, ...);
```

- **Formatted** input from a file and output to a file. They work **like scanf() / printf()**, but read from any stream rather than only stdin, and write to any stream rather than only stdout.
- `printf(...)` is the same as `fprintf(stdout, ...)`, and `scanf(...)` is the same as `fscanf(stdin, ...)`.
- Typical use: write error messages with `fprintf(stderr, ...)`, or read/write text files made of numbers and words.

Function	Return value
`fscanf`	Number of input items successfully assigned (may be less than expected), or EOF if input ended before the first conversion
`fprintf`	Number of characters written, or a negative value on error

Write then read formatted data

```
FILE *out = fopen("scores.txt", "w");
fprintf(out, "%d %.2f\n", 42, 97.5);
fclose(out);

FILE *in = fopen("scores.txt", "r");
int id; double score;
while (fscanf(in, "%d %lf", &id, &score) == 2)   // loop while both items read
    printf("id=%d score=%.1f\n", id, score);
fclose(in);
```

> **Watch out**
> With `fscanf` every argument after the format must be a **pointer** (`&id`), and `double` needs `%lf` for input although `%f` is used for output. Always compare the return value to the number of items you expected.

8. Binary I/O: fread and fwrite

Slide 14

```
size_t fread (void *ptr, size_t sz, size_t n, FILE *stream);
size_t fwrite(void *ptr, size_t sz, size_t n, FILE *stream);
```

- Read or write a **sequence of bytes** from/to a stream, as **n items each of size sz**, to/from the memory at `ptr`. No text conversion happens; the raw bytes of memory are copied.
- Both **return the number of items** (not bytes) read or written.
- If the return value is **smaller than n**, then end-of-file or an error occurred.

Example from the slide: write a whole 2-D array in one call

```
int    A[10][20];
size_t n = 10 * 20;
if (fwrite(A, sizeof(int), n, fp) != n) {
    // error
}
```

Here `sz` is `sizeof(int)` and `n` is 200, so 200 items of 4 bytes (typically) = 800 bytes are written. A 2-D C array is contiguous in memory, which is why one call can write all of it. Reading it back is symmetric:

```
size_t got = fread(A, sizeof(int), n, fp);
if (got != n) {
    if (feof(fp))        fprintf(stderr, "file too short\n");
    else if (ferror(fp)) perror("fread");
}
```

Watch out
Binary files are **not portable** between machines. Different integer sizes, byte order (endianness) and structure padding mean the same bytes can mean different values elsewhere. Also never fwrite a struct that holds pointers; the addresses are meaningless when read back later.

Good to know
Extra: `fread` and `fwrite` are far faster than looping over `fgetc`/`fputc` for large data, because they transfer a whole block per call.

9. Moving the file position indicator

Slide 15

```
long ftell(FILE *stream);
int  fseek(FILE *stream, off_t offset, int whence);
```

- `ftell` **reads** the current file position indicator (bytes from the start). Returns **-1** on error.
- `fseek` **sets** the file position indicator. Returns **0** on success and **-1** on error.
- The new position is `offset` bytes relative to `whence`:

whence	Offset is measured from
`SEEK_SET`	the beginning of the file
`SEEK_CUR`	the current position

whence	Offset is measured from
`SEEK_END`	the end of the file

Examples from the slide

```
fseek(fp,   0, SEEK_SET);   // move to the beginning
fseek(fp, 200, SEEK_CUR);   // move forward 200 bytes
fseek(fp,  -1, SEEK_END);   // move to the last byte
```

- **Not all streams are seekable.** A terminal or a pipe has no meaningful "position". The slide lists `errno: EBADF` as one error to expect; on a pipe or FIFO you will usually see `ESPIPE` (illegal seek). Always check the return value.

Common idiom (extra)

```
// get the size of a file in bytes
fseek(fp, 0, SEEK_END);
long size = ftell(fp);
fseek(fp, 0, SEEK_SET);        // back to the start
```

Good to know
Extra: you may seek past the end of a file and write there; the gap reads back as zero bytes. In append mode, writes always go to the end regardless of the position. `rewind(fp)` is shorthand for seeking to the start and clearing the error flags.

10. feof and fflush

Slide 16

```
// Check if end-of-file is set (after a read attempt!)
int feof(FILE *stream);

// Force write of buffered data
int fflush(FILE *stream);
```

feof

- Returns **non-zero** if the end-of-file indicator of the stream is set.
- The slide stresses **"after a read attempt!"**: the flag is only set once a read has actually tried to go past the last byte. Being positioned exactly at the end is not enough.

The classic feof bug

```
// WRONG: processes the last line twice (or garbage)
while (!feof(fp)) {
    fgets(line, sizeof line, fp);
    process(line);
}

// RIGHT: test the read itself
while (fgets(line, sizeof line, fp) != NULL) {
    process(line);
}
// afterwards, feof(fp) vs ferror(fp) tells you why the loop ended
```

fflush

- **Forces the write of buffered data** to the file (or screen) now, instead of waiting until the buffer is full or the file is closed.
- Returns 0 on success, EOF on error. This connects to slide 5: it is the cure for "printf output does not appear immediately".

```
printf("Enter a number: ");     // no newline, so may stay in the buffer
fflush(stdout);                 // make the prompt appear now
```

Good to know
Extra: `fflush(NULL)` flushes all open output streams. Flushing an *input* stream is not defined by the C standard, so do not rely on it to discard typed-ahead input.

The slide deck ends with two reminders: **Read the manual pages!** and **Check the return values!** Nearly every bug in this chapter comes from ignoring one of them.

11. Worked examples

Extra: combine the slide topics

Example A: copy a file (character I/O, error handling, open/close)

```
#include <stdio.h>

int copy_file(const char *src, const char *dst)
{
    FILE *in = fopen(src, "r");
    if (in == NULL) { perror(src); return -1; }

    FILE *out = fopen(dst, "w");
    if (out == NULL) { perror(dst); fclose(in); return -1; }

    int c, status = 0;
    while ((c = fgetc(in)) != EOF) {
        if (fputc(c, out) == EOF) { perror("fputc"); status = -1; break; }
    }
    if (ferror(in)) { perror("fgetc"); status = -1; }

    fclose(in);
    if (fclose(out) == EOF) { perror("fclose"); status = -1; }
    return status;
}
```

Example B: count lines and characters (line I/O)

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[])
{
    if (argc != 2) { fprintf(stderr, "usage: %s file\n", argv[0]); return 1; }
    FILE *fp = fopen(argv[1], "r");
    if (fp == NULL) { perror(argv[1]); return 1; }

    char line[256];
    int lines = 0; long chars = 0;
    while (fgets(line, sizeof line, fp) != NULL) {
        lines++;
        chars += strlen(line);
    }
    printf("%d lines, %ld chars\n", lines, chars);
    fclose(fp);
    return 0;
}
```

Example C: save and reload an array (binary I/O, seeking)

```
int data[100];
/* ... fill data ... */

FILE *fp = fopen("data.bin", "w+");          // read+write, truncate
if (fp == NULL) { perror("data.bin"); return 1; }

if (fwrite(data, sizeof(int), 100, fp) != 100) { perror("fwrite"); }

fseek(fp, 50 * sizeof(int), SEEK_SET);       // jump to element 50
int x;
if (fread(&x, sizeof(int), 1, fp) == 1)
    printf("data[50] = %d\n", x);

fclose(fp);
```

Good to know
Switching between reading and writing on the same stream ("r+", "w+", "a+") requires an `fseek` or `fflush` in between, according to the C standard. Forgetting this can produce odd behaviour.

12. Pitfalls, quick reference, self-check

Extra

Common pitfalls

Mistake	Why it is wrong / fix
Not checking fopen's result	NULL pointer then crashes. Always test for NULL and call perror.
char c = fgetc(fp)	EOF cannot be detected reliably. Use int.
while (!feof(fp))	The flag is set only after a failed read. Test the read function's return value.
gets(buf)	No bounds checking (buffer overflow). Use fgets(buf, sizeof buf, stdin).
Opening with "w" to add data	Truncates the file. Use "a" to append.
Forgetting fclose	Leaks resources and may lose buffered output on a crash.
Reading errno without a failure	Value is only meaningful right after a call that reported failure.
Comparing fread/fwrite result to bytes	They return the number of **items**, not bytes.
Assuming printf output appears instantly	It is buffered. Use fflush(stdout) or print a newline to a terminal.
Passing a value instead of a pointer to fscanf	Use &variable.; Check the return value equals the number of items expected.

Quick reference

Function	Purpose	Returns on success	On failure
`fopen`	open a file	FILE *	NULL, errno set
`fclose`	close a stream	0	EOF, errno set
`fgetc / getc / getchar`	read one char	char as int	EOF
`fputc / putc / putchar`	write one char	the char written	EOF
`ungetc`	push back one char	the char	EOF
`fgets`	read a line (max size-1)	buf	NULL
`fputs`	write a string	non-negative (slide: count)	EOF
`fscanf`	formatted read	items assigned	EOF / fewer items
`fprintf`	formatted write	chars written	negative
`fread / fwrite`	binary block I/O	items transferred (n)	count less than n
`ftell`	get position	position	-1
`fseek`	set position	0	-1
`feof`	test EOF flag	non-zero if set	-
`fflush`	flush buffer	0	EOF
`perror`	print errno message	(void)	-

Self-check questions

1. Why does fgetc return int instead of char?

Answer: So that EOF (a negative value) can be told apart from every valid byte value 0-255.

2. What is the difference between modes "w+" and "a+"?

Answer: "w+" truncates the file; "a+" keeps the contents, reads start at the beginning, and every write goes to the end.

3. fgets(buf, 10, fp) reads a line of 30 characters. What happens?

Answer: It reads at most 9 characters, adds '\0', and the rest of the line is returned by later calls.

4. Why can printf output appear late?

Answer: stdio buffers output; it is sent when the buffer fills, on a newline to a terminal, on fflush, or when the program ends.

5. What does fwrite(A, sizeof(int), n, fp) != n tell you?

Answer: Fewer than n items were written, so an error occurred.

6. Which call moves to the last byte of a file?

Answer: fseek(fp, -1, SEEK_END).

7. When is feof true?

Answer: Only after a read attempt has tried to read past the end of the file.

13. Slide-by-slide coverage checklist

Use this to confirm that nothing on the slides is missing from your notes

Slide	Title	Covered in
1	A C Primer (9): I/O and Files	Title page
2	errno	Section 1
3	Files and directories	Section 2
4	The stdio library	Section 3
5	Files and I/O API	Section 3
6	File as stream of bytes	Section 3 (with diagram)
7	Opening Streams	Section 4
8	Closing Streams	Section 4
9	fgetc / fputc	Section 5
10	getc / putc and ungetc	Section 5
11	getchar / putchar	Section 5
12	More than one byte: get a line	Section 6
13	Formatted output	Section 7
14	For binary data	Section 8
15	Moving file position indicator	Section 9
16	More useful stdio functions	Section 10